Introduction to ActorForth

A language where the stack is the state, types are the syntax, and actors are the concurrency.

In the style of Leo Brodie's "Starting FORTH" and "Thinking FORTH"

Preface: Why ActorForth Exists

Language design has always been one of my most favorite intellectual pastimes. I chew on a problem, imagine a language in which the solution would be easy to read, and iterate until the shape is elegant. Formally you might call this *creating an ontology for the problem domain and designing a language that expresses it well.* That language is usually unsuitable for anything else — and that's typically where the exercise ends, because making it real is too expensive to justify. So you use it as a specification and write the program in C++ or Erlang or Python instead.

Forth is what happens when you refuse to accept that cost. Chuck Moore's answer to the "too expensive to build your own language" problem was to build a language-building tool so small and direct that you build your domain language *in front of yourself,* at the REPL, in minutes rather than months. The cost collapses to near zero. But Forth paid for that with famously minimal guard rails — no type system, no concurrency story — and got a reputation for being a write-only language.

ActorForth is Forth with those two holes filled:

1. **A type system that does the bookkeeping,** so you don't have to remember what's on the stack at run time.
2. **Actors as a primitive,** so the unbounded non-determinism of real-world systems — the kind you encounter the moment your code touches the outside world — stops being a footnote and becomes the concurrency model itself.

Everything else is in service of one idea: *programming in ActorForth means building a small language tailored to your problem, and then writing your problem in that language.* That's why the REPL is central, why types own their dispatch, why the compiler is itself four ordinary types with handlers. The machinery of making a DSL real is the machinery you already use.

You may not love the RPN notation. You don't have to. The aesthetic is a means, not the point. *The best ActorForth programs rarely use ActorForth primitives* — they use the vocabulary the programmer defined for their domain, in the same spirit that the best C++ applications rarely use the built-in types.

Giants this stands on. Chuck Moore for the engineering answer that language creation can be cheap. Carl Hewitt, whose Actor Model is the only concurrency story I've found that survives contact with unbounded non-determinism in the real world. Guy Steele for the idea that a language should be small and growable, not large and finished. Bjarne Stroustrup for building the vocabulary of zero-overhead abstractions that user-defined types can replace the built-ins without a runtime tax. Evan Czaplicki for showing that a small, opinionated language can still be a joy to write in. Margaret Hamilton and Edsger Dijkstra for establishing, at life-critical scale, what it means to demonstrate correctness — while also marking the boundary where formal verification stops scaling and *simplicity* has to take over. On that last point the ActorForth stance (set out in the 2019 architectural drivers paper) compresses to:

> *A system should be expressed so simply that its correctness is obvious to the reader.*

Not Hoare exactly — but the spirit runs through his Turing lecture too, and through De Millo, Lipton, and Perlis on *Social Processes and Proofs*, and through Einstein's "as simple as possible, but no simpler." The architectural-drivers document (`docs/architectural-drivers-considerations.md`) carries the formal version of this argument. What follows here is the practical one.

A note on HOS. Some later chapters use Hamilton's *Higher Order Software* (HOS) vocabulary: subsystems, transitions, axioms, effects. HOS is a design discipline for hierarchies of state machines with formally checked composition — the methodology Hamilton developed after Apollo. Its value proposition is narrow but sharp: explicit, checked, hierarchical state machines where safety properties emerge from the axiom set. That is what you want for lifts, payment terminals, signalling, medical devices, protocol state. It is not what you want for CRUD, aggregation, ML inference, games, or most web backends. System-of-Systems is more universal in the sense that the generic actor model covers far more ground — but HOS is not the actor model; it is a discipline on top of it. ActorForth provides HOS as an optional type-backed DSL, not a tax on every program you write. If your problem isn't state-machine-heavy, you can skip the HOS chapters.

Now — the stack.

Chapter 1: The Stack Is Everything

If you've used Forth before, you know the feeling: there are no variables cluttering up the place, no assignment

statements, no declarations. There's just the stack, and the words that transform it. ActorForth inherits this directness, but adds something Forth never had — every value on the stack knows what it is.

In traditional Forth, a number is just bits. The programmer remembers what those bits mean. In ActorForth, there are no naked values. Everything on the stack is a typed pair: a type tag and a value, married together for life. When you type:

```
42
```

The interpreter tries to find an operation named `42`. No dictionary has one. Before giving up, it asks each type: "Do you want to claim this token?" The Int type's literal handler recognizes `42` as an integer and pushes `{Int, 42}` onto the stack. The type stays with the value everywhere it goes.

You can also be explicit:

```
42 int
```

Here `42` lands on the stack as an Atom — just text that didn't match any operation or literal handler. Then `int` sees that Atom, converts it to a proper integer, and puts `{Int, 42}` on the stack. Both forms produce the same result. The explicit form matters when literal handlers can't guess what you want, or when you want to be clear about your intent.

This isn't just bookkeeping. The type of the value on top of the stack determines what every subsequent word *means*. This is the central idea of ActorForth, and everything else follows from it.

Your First Session

Start the REPL:

```
$ rebar3 shell
1> af_repl:start().
ActorForth REPL. ^C to exit.
ok:
```

That `ok:` prompt is waiting. Let's put some numbers on the stack:

```
ok: 10 20
```

The stack now holds two integers — both claimed by the Int literal handler. We can see the stack:

```
ok: stack
0) 20 : Int
1) 10 : Int
```

The top of stack (TOS) is shown first — `20` is on top, `10` is underneath. Now let's add them:

```
ok: +
ok: stack
0) 30 : Int
```

The `+` word consumed both integers and left their sum. Notice we didn't write `10 + 20`. There is no infix notation. The values go on the stack first, then the operation acts on them. This is postfix — also called Reverse Polish Notation — and it eliminates the need for parentheses, operator precedence, or parsing ambiguity.

Stack Manipulation

A handful of words handle stack shuffling:

```
ok: 5 dup
```

`dup` copies the top item. The stack now has two copies of `{Int, 5}`.

```
ok: stack
0) 5 : Int
1) 5 : Int
```

```
ok: drop
```

`drop` discards the top item. Back to one `5`.

```
ok: 3 swap
```

`swap` exchanges the top two items. Now `5` is on top and `3` is underneath. Or rather — `3` was on top after pushing it, then `swap` put `5` back on top:

```
ok: stack
0) 5 : Int
1) 3 : Int
```

`rot` rotates the third item to the top:

```
ok: 1 2 3
ok: stack
0) 3 : Int
1) 2 : Int
2) 1 : Int
ok: rot
ok: stack
0) 1 : Int
1) 3 : Int
2) 2 : Int
```

There's also `over`, which copies the second item to the top, and `2dup`, which copies the top *two* items.

These words — `dup`, `drop`, `swap`, `rot`, `over`, `2dup` — work on any type. They're in the global dictionary because they don't care what's on the stack. An Int, a Bool, a user-defined Point — all the same to `swap`.

Chapter 2: Types Drive Everything

Here is the key insight that separates ActorForth from Forth, and indeed from most languages: **the type on top of the stack determines the dictionary that the next word is looked up in.**

When you type `+` and TOS is an Int, the interpreter finds `+` in the Int dictionary. If TOS were something else — a Bool, say — the interpreter wouldn't find `+` there, and would fall back to the global (Any) dictionary, and if it's not there either, `+` would just become an Atom on the stack. No crash. No syntax error. Just an Atom.

This means the *same word* can mean completely different things depending on what's on the stack. The stack is not just data storage — it's the program's state machine.

The Dispatch Rules

When the interpreter encounters a token, it follows these steps:

1. **Search TOS type's dictionary.** If TOS is `{Int, 42}`, look in Int's dictionary.
2. **Check TOS type's handler.** Some types (like compiler states) intercept all tokens.
3. **Search the Any dictionary.** Global operations live here.
4. **Try literal handlers.** Each type gets a chance to claim the token.
5. **Push as Atom.** Nothing matched — it's just text. Put `{Atom, "whatever"}` on the stack.

That's the entire interpreter. Five steps. It never changes. What changes is what's on the stack, and therefore which dictionary is consulted first.

Constructors: The Gateway Between Types

Every type needs a way to create instances. The convention is: the constructor is a lowercase version of the type name, and it converts from some input type.

```
ok: 42 int         # Atom -> Int (explicit constructor)
ok: True bool      # Atom -> Bool (explicit constructor)
```

Constructors live in the Any dictionary because they need to be reachable regardless of what's currently on the stack. The input type constraint ensures they only fire when the right type is available.

For common types, literal handlers provide a shorthand:

```
ok: 42             # Literal handler claims it -> {Int, 42}
ok: True           # Literal handler claims it -> {Bool, true}
ok: False          # Literal handler claims it -> {Bool, false}
```

Both forms are valid. The explicit constructor is still there when you need it — for example, when a token could be ambiguous, or when you're passing a string that happens to look like a number.

Arithmetic and Comparison

Integer operations live in the Int dictionary:

```
ok: 10 3 -         # 10 - 3 = 7
ok: 7 6 *          # 7 * 6 = 42
ok: 100 3 /        # 100 div 3 = 33  (integer division)
```

Arithmetic follows the Forth convention: the *second* item on the stack is the left operand. So `10 3 -` computes `10 - 3`, not `3 - 10`. Think of it as: "put 10 down, put 3 on top, subtract the top from what's underneath."

Comparisons produce Booleans:

```
ok: 10 20 <        # Is 10 < 20?  -> {Bool, true}
ok: 5 5 ==         # Is 5 == 5?   -> {Bool, true}
ok: 3 7 >=         # Is 3 >= 7?   -> {Bool, false}
```

Boolean operations live in the Bool dictionary:

```
ok: True not       # -> {Bool, false}
```

No Syntax Errors, Only Type Errors

If you type something the interpreter doesn't recognize — and no literal handler claims it — it simply goes on the stack as an Atom:

```
ok: hello world foo
ok: stack
0) foo : Atom
1) world : Atom
2) hello : Atom
```

Three Atoms, sitting peacefully on the stack. They're not errors — they're data. Maybe some future word will know what to do with them. Maybe they'll be arguments to a constructor. Maybe they're names being captured by the compiler. The interpreter doesn't judge.

This is a fundamental philosophical point: **ActorForth has no syntax to violate.** There is only the question of whether the types on the stack match what an operation expects. Everything is a type error or a successful dispatch.

Chapter 3: Defining New Words

Here's where ActorForth starts to feel powerful. You can define new words — named operations with typed signatures — and the system is smart enough to put them in the right dictionary automatically.

The Colon Definition

The syntax mirrors traditional Forth, extended with type signatures:

```
: word-name  InputTypes -> OutputTypes ; body .
```

Let's define a word that squares an integer:

```
ok: : square Int -> Int ; dup * .
```

Read it aloud: "Define `square`. It takes an Int, returns an Int. The body is: duplicate the top, then multiply." The `.` at the end closes the definition.

Now use it:

```
ok: 7 square
ok: stack
0) 49 : Int
```

Because the input type is `Int`, `square` was automatically registered in the Int dictionary. The interpreter finds it by the same mechanism it finds `+` — TOS is Int, search Int's dictionary.

Building Words from Words

Words can call other words:

```
: double Int -> Int ; dup + .
: quadruple Int -> Int ; double double .
```

`quadruple` calls `double` twice. Each call does the full dispatch — TOS is Int, find `double` in Int's dictionary, execute it. No special calling convention.

```
ok: 3 quadruple
ok: stack
0) 12 : Int
```

Words with Multiple Inputs

A word can consume several stack items:

```
: add3 Int Int Int -> Int ; + + .
```

This takes three Ints from the stack and produces their sum:

```
ok: 10 20 30 add3
ok: stack
0) 60 : Int
```

Where Words Live

A word's home dictionary is determined by the rightmost input type — the type that will be on top of the stack when the word is called. This happens automatically:

- `: square Int -> Int ;` — lives in Int dict (TOS will be Int)
- `: origin -> Point ;` — lives in Any dict (no inputs)
- `: add Counter Int -> Counter ;` — lives in Int dict (TOS will be Int)

You never specify which dictionary to use. The type signatures do it for you.

How Compilation Works

Here's the beautiful part. When you type `:`, it pushes a `WordDefinition` value onto the stack. Now the TOS type is `WordDefinition`, so the interpreter's dictionary context changes. The next token — the word name — is captured by WordDefinition's handler instead of being executed or pushed as an Atom.

Then the TOS type becomes `InputTypeSignature`. Now every token is interpreted as a type name and accumulated into the input signature. When `->` appears, it's found in InputTypeSignature's dictionary. It transitions the TOS to `OutputTypeSignature`.

Same pattern continues: type names accumulate into the output signature. `;` transitions to `CodeCompile`. Now tokens are recorded as operation references — the body of the word. Finally `.` finishes compilation, saves the word, and pops all the compiler state off the stack.

The interpreter never changed. It's still doing the same dispatch it always does. The compiler is just four types — `WordDefinition`, `InputTypeSignature`, `OutputTypeSignature`, `CodeCompile` — each with a tiny dictionary. The types on the stack redirected what the tokens meant.

This is why the compiler is so small. It's not a separate system. It's just more types.

Chapter 4: Thinking in Patterns

Traditional Forth — and most languages — uses `if` / `else` for branching. ActorForth takes a different path. The preferred control flow mechanism is **pattern matching through overloaded word definitions.**

Value Constraints

The dispatch mechanism doesn't just match on type — it can match on specific values within a type. A word whose signature says `0` for an Int parameter matches only when that parameter is exactly zero. The general signature `Int` matches any integer. When both are registered under the same name, the more specific value-constrained version is tried first.

Fibonacci: The Canonical Example

Here's the actual `fib.a4` from the samples directory:

```
: fib Int -> Int ;
    : 0 -> 0 ;
    : 1 -> 1 ;
    : Int -> Int ;
        dup
        1 -
        fib
        swap
        2 -
        fib
        +.
```

The master signature `: fib Int -> Int ;` establishes the word name and provides type context. Inside it, three sub-clauses handle different cases:

- `: 0 -> 0 ;` — when the input is exactly 0, return 0
- `: 1 -> 1 ;` — when the input is exactly 1, return 1
- `: Int -> Int ;` — for any other Int, compute `fib(n-1) + fib(n-2)`

No `if` statement. No `else`. The dispatch *is* the branch. Each case reads independently, and adding a new case means adding a new sub-clause — you don't modify existing code.

Test it:

```
ok: 0 fib 0 assert-eq
ok: 1 fib 1 assert-eq
ok: 6 fib 8 assert-eq
ok: 10 fib 55 assert-eq
```

Recursive Countdown

Pattern matching naturally expresses recursion with a base case:

```
: countdown Int -> ;
    : 0 -> ; drop
    : Int -> ; 1 - countdown.
```

Two sub-clauses: when the value is 0, drop it and stop. For any other Int, subtract 1 and recurse. The BEAM VM handles tail recursion efficiently, so this is as fast as a loop.

Why Pattern Matching?

Three reasons:

Clarity. Each case is a separate clause. You can read them independently. There's no nested `if` / `else` to untangle.

Extensibility. Adding a new case means adding a new clause. You don't modify existing code. This is the Open/Closed Principle, enforced by the language.

BEAM compatibility. When ActorForth compiles to BEAM bytecode, pattern-matched word definitions map directly to Erlang's function clause mechanism. The BEAM VM is *built* for this — it compiles pattern matches into efficient jump tables. We get that optimization for free.

Pattern Matching on Booleans

Pattern matching isn't limited to integers. Booleans work the same way — `True` and `False` are value constraints:

```
: to-int Bool -> Int ;
    : True  -> Int ; drop 1
    : False -> Int ; drop 0 .
```

This is ActorForth's answer to `if` / `else`. The `drop` removes the matched boolean value from the stack. More complex branching is just more sub-clauses:

```
: choose Int Int Bool -> Int ;
    : True  -> Int ; drop drop
    : False -> Int ; drop swap drop .
```

This `choose` word takes two Ints and a Bool. If True, keep the deeper Int (the "then" value); if False, keep the TOS Int (the "else" value). Notice that sub-clauses with fewer elements than the master signature automatically right-align — `True` matches the Bool position (rightmost/TOS), and the Int positions default to type-only matching.

Pattern Matching on Strings

Quoted strings can also be value constraints:

```
: greet String -> String ;
    : "hello"   -> String ; drop "Hello World!"
    : "goodbye" -> String ; drop "Farewell!"
    : String    -> String ; .

ok: "hello" greet
ok: stack
0) "Hello World!" : String
```

The general `String` clause acts as the fallback — unmatched strings pass through unchanged.

Composition Over Branching

In ActorForth, we compose small words rather than writing long procedures with branches:

```
: double Int -> Int ; dup + .
: square Int -> Int ; dup * .
: bump Int -> Int ; 1 + .
```

Each word does one thing. Combine them as needed:

```
ok: 5 double bump square
ok: stack
0) 121 : Int
```

That's `((5 * 2) + 1)^2 = 121`. No temporary variables, no intermediate assignments. The data flows through the words.

Chapter 5: Product Types — Building Data Structures

Numbers and booleans are fine, but real programs need structured data. ActorForth provides **product types** — named records with typed fields.

Defining a Type

```
type Point
    x Int
    y Int .
```

This registers a new type called `Point` with two fields: `x` (an Int) and `y` (an Int). The system auto-generates three things:

1. **A constructor** — `point` (lowercase), registered in the Any dictionary

2. **Getters** — `x` and `y`, registered in the Point dictionary
3. **Setters** — `x!` and `y!`, for updating fields

Put each field name and its type on its own line. It makes it easy for the eye to parse what's a name and what's a type. The syntax is the same either way — `type Point x Int y Int .` works too — but the vertical layout is clearer.

Construction

The constructor takes field values from the stack in definition order:

```
ok: 10 20 point
ok: stack
0) {x: 10, y: 20} : Point
```

First the x value, then the y value, then `point` to assemble them.

Field Access

Getters are **non-destructive** — they push the field value while leaving the instance on the stack:

```
ok: 10 20 point
ok: x
ok: stack
0) 10 : Int
1) {x: 10, y: 20} : Point
```

This means you can access multiple fields without `dup`:

```
ok: 10 20 point
ok: x swap y
ok: stack
0) 20 : Int
1) {x: 10, y: 20} : Point
2) 10 : Int
```

When you're done with the instance, `swap drop` cleans it up:

```
ok: 10 20 point x swap drop
ok: stack
0) 10 : Int
```

Auto-Field Bindings Inside Word Bodies

The `x` / `y` access above works at the REPL, but inside a word definition there's a cleaner form. When a word's signature declares a product-type input, its field names become **scoped bindings** in the body.

```
: perimeter Point -> Int ;
    x 2 * y 2 * +
    swap drop
.
```

Here `x` and `y` inside the body don't fire the getters — they directly push the field values that were on the Point at the moment the word was called. The instance stays on the stack, so `swap drop` at the end still applies; the bindings are a cleaner path to the same stack state.

Positional access handles words that take more than one product:

```
: distance Point Point -> Int ;
    ..x .x - dup *
    ..y .y - dup * +
    swap drop swap drop
.
```

Read the dot count left-to-right against the signature: `.x` is the field `x` of the **first** `Point` in the signature (source-leftmost), `..x` is the second, `...x` the third, and so on. Dot count matches slot position. This is both clearer and more reliable than stack-manipulation tricks for multi-argument access.

Bare field names are available when unambiguous — if the signature has one Point and one Color, `x` is unambiguously Point.x and `r` is unambiguously Color.r. If the signature has two Points, bare `x` would be ambiguous and the compiler declines to bind it; use `.x` and `..x` instead.

Field bindings are a **snapshot** taken at word entry. If the body updates a field via a setter, the binding still holds the original value. For most code this is what you want — immutable semantics everywhere — but it's worth knowing.

Updating Fields

Setters use the `!` suffix convention (read "store", as in Forth tradition). Push the new value, then apply the setter:

```
ok: 10 20 point     # create Point(10, 20)
ok: 99 x!           # update x to 99
ok: x swap y
ok: stack
0) 20 : Int
1) {x: 99, y: 20} : Point
2) 99 : Int
```

The setter consumes the new value and the Point, then pushes a new Point with the updated field. The original is not mutated — there is no mutation in ActorForth. Every "update" creates a new value.

Types Compose with Words

Define operations that work with your types:

```
type Point
    x Int
    y Int .

: origin -> Point ; 0 0 point .
: move-right Point -> Point ; x 1 + x! .
```

`origin` takes no inputs and produces a Point at (0, 0). `move-right` takes a Point, reads `x` (non-destructive, leaves the Point), adds 1, then stores back with `x!`. No `dup` gymnastics needed — the getter leaves the instance for the setter.

Types with More Fields

Product types can have any number of fields with mixed types:

```
type Account
    balance Int
    ledger List .
```

Fields can be of any registered type — Int, Bool, List, or even another product type.

Why Product Types Matter

Product types make ActorForth a viable language for real programs. You're not limited to loose values on the stack — you can bundle related data together, name the fields, and define operations that work on the bundle. And because the type goes on the stack, it participates in dispatch just like Int or Bool. When a Point is on top, Point-specific words become available. The stack is still the state machine.

Product types are also the foundation for actors. Any product type instance can become an actor's state — and the operations defined on that type become the actor's interface.

Chapter 6: Lists

ActorForth provides a built-in List type that wraps native Erlang cons cells. This isn't an accident — BEAM lists are already singly-linked cons cells with O(1) prepend, which is exactly what a stack-based language needs.

Building Lists

`nil` pushes an empty list. `cons` prepends an item:

```
ok: nil
ok: stack
0) [] : List
```

```
ok: 1 cons 2 cons 3 cons
ok: stack
0) [3, 2, 1] : List
```

Each `cons` takes an item and a list, and produces a new list with the item on front. Building left-to-right with cons means the last item consed is the first in the list.

Decomposing Lists

`head` returns the first element. `tail` returns everything after the first:

```
ok: nil 1 cons 2 cons 3 cons
ok: dup head
ok: stack
0) 3 : Int
1) [3, 2, 1] : List

ok: drop tail
ok: stack
0) [2, 1] : List
```

`length` tells you how many elements:

```
ok: nil 1 cons 2 cons 3 cons length
ok: stack
0) 3 : Int
```

Lists Are Heterogeneous

Any stack item can be consed onto any list. The items keep their original types:

```
ok: nil 42 cons True cons
ok: stack
0) [True : Bool, 42 : Int] : List
```

This matches Erlang's own list semantics. Each element is a full `{Type, Value}` pair.

Bracket Literals

For straight list construction, the `nil X cons Y cons ...` idiom is noisy and its reversed visual order (last-written-first-in-list) trips people up. ActorForth accepts `[ ... ]` as syntactic sugar:

```
ok: [ 1 2 3 ]
ok: stack
0) [1, 2, 3] : List

ok: [ ] length 0 assert-eq
ok: [ 1 [ 2 3 ] 4 ] length 3 assert-eq
```

Brackets are just parser-level sugar: `[` pushes a sentinel and `]` collects everything back to it into a List in source order. Items between the brackets are evaluated as normal stack code, so `[ 1 2 + 10 ]` yields `[3, 10]`. Nesting works because each `[` pushes its own marker.

Use whichever you prefer — brackets for literal construction, `cons` when you're prepending to an existing list. The two produce identical values when the items are literal; they differ only in visual ordering:

```
[ 1 2 3 ]                       # -> [1, 2, 3]
nil 3 cons 2 cons 1 cons        # -> [1, 2, 3] (same value, different writing direction)
```

Lists in Practice

Lists become powerful when combined with product types. Here's a ledger that tracks transactions:

```
type Transaction
    kind Atom
```

```
    amount Int .

# Build a list of transactions
nil
Deposit 500 transaction cons
Withdrawal 100 transaction cons
```

The list holds typed values — each element is a full Transaction, carrying its type information along.

Chapter 7: Actors — Concurrency as a Primitive

Most languages bolt concurrency on as an afterthought — threads, locks, mutexes, all the things that make parallel programming a nightmare. ActorForth takes a different approach: concurrency is built in from the beginning, using the actor model, and it maps directly to Erlang's battle-tested process system.

An actor is a process with its own stack, its own state, and a mailbox. Actors communicate by sending messages. There is no shared state. No locks. No races. If you need another actor to do something, you send it a message.

> **Aside — what Alan Kay actually meant by "object-oriented."**
>
> Kay coined the phrase for Smalltalk and has spent forty years pointing out that it did *not* mean classes, inheritance, or `obj.field`-style direct access. His definition:
>
> > *"OOP to me means only messaging, local retention and protection and hiding of state-process, and extreme late-binding of all things."*
>
> That is a description of an actor. A process with private state that responds only to messages, whose dispatch is resolved at the moment the message arrives. Most languages people call "object-oriented" — Java, C#, mainstream C++ — are class-oriented: methods called through vtables, state reached by direct memory access, dispatch resolved mostly at compile time. Kay has said more than once that he regrets the label being attached to that lineage.
>
> When you write `<< withdraw 50 >>` against a Counter or a BankAccount actor later in this chapter, you are doing the thing Kay was describing. The `server` you just built is a real BEAM process; the message crosses a real process boundary; the dispatch is resolved on receipt through the type's dictionary. Nothing else in mainstream industry programming reaches Kay's definition this literally. See *Part 0.6* in `architectural-drivers-considerations.md` for the argument in full.

The `server` Pattern

The `server` word takes any typed instance from the stack and spawns an Erlang process to hold it. That process becomes a stateful actor — it holds the instance as its state, and accepts messages that correspond to user-defined words operating on that type.

Let's build a counter:

```
type Counter
    value Int .

: increment Counter -> Counter ;
    dup value 1 + value!.

: count Counter -> Counter Int ;
    dup value.

0 counter server
```

That last line does three things: pushes `{Int, 0}`, calls `counter` to build `{Counter, #{value => {Int, 0}}}`, then `server` spawns a process to hold it. The stack now has an Actor reference.

Sending Messages with `<< >>`

The `<<` word enters send mode. Inside the block, tokens that match the actor's **vocabulary** are dispatched to the actor. Other tokens execute locally. `>>` exits send mode.

```
<< increment >>
<< increment >>
<< increment >>
<< count >> 3 assert-eq
```

Three increments, then a count. But notice something: `increment` and `count` behave differently:

- `increment` takes `Counter -> Counter` — same type in, same type out. No extra return values. This is a **cast** — fire-and-forget, asynchronous.
- `count` takes `Counter -> Counter Int` — it returns an extra Int beyond the state. This is a **call** — synchronous, waits for the reply.

The actor system classifies each word automatically based on its signature. You don't specify cast vs call — the types tell the story.

After the `<< count >>` call, the Int result (`3`) is pushed onto *your* stack, on top of the Actor reference. That's why `3 assert-eq` works — it pops the 3 and compares it.

What's In the Vocabulary?

The actor's vocabulary is built automatically from user-defined words whose signature includes the actor's state type. But **auto-generated ops are excluded**. The getter `value` and setter `value!` that the type system created? Not in the vocabulary. The actor's internal structure is private.

This is state encapsulation, enforced by the language. Want to read the counter's value? Define a word for it (`count`). Want to modify it? Define a word for it (`increment`). The auto-generated accessors work *inside* those word bodies — they execute within the actor process, where the state is on the stack. But from outside, you go through the public interface.

Local vs Remote Execution

Inside a `<< >>` block, not everything goes to the actor. Only vocabulary words are dispatched remotely. Everything else runs locally:

```
<< 5 add >>
```

Here `5` is a literal — it executes locally, pushing `{Int, 5}` onto a temporary local stack. Then `add` (if it's in the actor's vocabulary) dispatches to the actor with that `5` as an argument.

This is how you pass arguments to actor operations. Compute them locally, then the vocabulary word scoops them up and sends them along.

Stopping an Actor

Every actor has a built-in `stop` word:

```
<< stop >> drop
```

`stop` terminates the actor process and pushes its final state onto your stack (the `drop` discards it). After `stop`, the Actor reference is no longer valid.

A Real Example: Bank Account

Here's the full bank actor from `samples/bank_actor.a4`:

```
type Transaction
    kind Atom
    amount Int .

type Account
    balance Int
    ledger List .

# Helper: build a Transaction
: make-tx Atom Int -> Transaction ;
    transaction.

# Helper: prepend a Transaction onto Account's ledger
: add-tx Account Transaction -> Account ;
    swap dup ledger rot cons ledger!.

# Helper: add to Account balance
: credit Account Int -> Account ;
```

```
    swap dup balance rot + balance!.

# Helper: subtract from Account balance
: debit Account Int -> Account ;
    swap dup balance rot - balance!.

: deposit Account Int -> Account ;
    dup Deposit swap make-tx rot swap add-tx swap credit.

: withdraw Account Int -> Account ;
    dup Withdrawal swap make-tx rot swap add-tx swap debit.

: get-balance Account -> Account Int ;
    dup balance.

: get-ledger Account -> Account List ;
    dup ledger.

: tx-count Account -> Account Int ;
    dup ledger length.

# Open account with $1000 and empty ledger
1000 nil account server

# Deposits
<< 500 deposit >>
<< 250 deposit >>

# Withdrawal
<< 100 withdraw >>

# Check balance: 1000 + 500 + 250 - 100 = 1650
<< get-balance >> 1650 assert-eq

# Check transaction count
<< tx-count >> 3 assert-eq

# Clean up
<< stop >> drop
```

Notice the helper words: `make-tx`, `add-tx`, `credit`, `debit`. Each does one thing. The higher-level words (`deposit`, `withdraw`) compose them. This is Thinking Forth — decompose until each word is small and clear.

Also notice the stack manipulation. Words like `credit` start with `swap dup balance rot` — that's three stack shuffling words before you get to the business logic (`+`). This is the honest cost of explicit stack threading with destructive getters. It works, it's correct, but it pushes the developer's mind away from the problem domain and into the mechanics of stack ordering. Future language improvements (non-destructive getters, possibly a context register) aim to reduce this friction.

Why Actors Belong in a Forth

The actor model and Forth share a philosophy: **keep things small and independent.** A Forth word does one thing. An actor handles one concern. Words compose by piping data through the stack. Actors compose by passing messages through mailboxes.

And because ActorForth runs on the BEAM, you inherit decades of engineering around reliable concurrent systems. Erlang processes are cheap — you can spawn millions of them. They're isolated — one crashing doesn't take down the others. They're distributable — they can run on different machines transparently.

ActorForth doesn't try to reinvent this. It exposes it directly, with stack-friendly syntax.

`self`

The `self` word pushes the current process's actor reference:

```
ok: self
ok: stack
0) <0.85.0> : Actor
```

This is useful for actors that need to send messages to themselves, or pass their own reference to other actors.

Chapter 8: Testing Your Code

ActorForth's test story has two layers. The bottom layer is two inline assertion words, already part of the core language. The upper layer is a *test DSL* — itself written in ActorForth — that turns inline assertions into named tests, groups, parallel actors, coverage, and structured output.

We'll start with the inline assertions, because the DSL is built on top of them.

The two assertion words

`assert` pops a `Bool`. If it's `True`, nothing happens. If it's `False`, execution stops with an error that names the file, line, and column:

```
ok: 5 3 > assert         # passes — 5 > 3 is true
ok: 3 5 > assert         # FAILS — 3 > 5 is false
Error: assertion_failed at stdin:1:7
  Assertion failed: expected True on stack
  Stack(1): False:Bool
```

`assert-eq` pops two values. If they're equal (same type and value), nothing happens. Otherwise the error shows what was expected and what was actually there:

```
ok: 7 square 49 assert-eq     # passes — 49 == 49
ok: 7 square 50 assert-eq     # FAILS
Error: assert_eq_failed at stdin:1:14
  Expected 50 but got 49
```

That's enough to sanity-check anything. Every sample file under `samples/` uses assertions as self-tests — run the file and silence means success.

The test DSL

Inline assertions are great while you're hacking. When you have hundreds of them, you want structure: names, grouping, per-test isolation, parallel execution, coverage. ActorForth ships a DSL for this, written *in ActorForth itself*, under `lib/testing/`. It's the canonical example of the DSL-first workflow that language is for.

Here's a complete test file:

```
# lib/testing/list_ops.test.a4 — excerpt

"cons chains build list" test :
    nil 1 cons 2 cons 3 cons
    dup length 3 assert-eq
    head 3 assert-eq
;

"head on empty raises" test-raises empty_list :
    nil head
;
```

The `.test.a4` extension tells the test runner "this file contains tests." The filename, with the extension stripped, becomes the implicit outer group — so the test above shows up in the report as `list_ops/cons chains build list`.

The vocabulary in brief

word	shape	what it does
`<atom> group … .`	group with an atom name, closed with `.`	open a scope
`<atom> group-serial … .`	opt out of parallel execution within this group	
`"name" test :` … `;`	positive test	body runs; silence = pass
`"name" test-raises Kind :` … `;`	negative test	body must raise `Kind`
`"name" "reason" skip-test :`	registered but skipped	emitted as `skip` with the

`... ;`	registered but skipped	reason
`"name" test-compiled :` ... `;`	flag body for native compilation (speed)	
`setup :` ... `;`	fixture before each test in the group	
`teardown :` ... `;`	fixture after each test	
`"label" tag`	attach tag to next test (accumulating)	
`N max-depth`	auto-assert peak data stack depth ≤ N	

Names are atoms for groups (short identifier) and strings for tests (descriptive prose). The type system enforces that — it's a design choice, not a stylistic convention. See the architectural-drivers doc for the reasoning.

Running tests

From the project root:

```
rebar3 shell --eval 'af_test_runner:run(["lib/testing"]).'
```

or, once you've built a release:

```
a4c test
```

Three output modes:

- **`--dashboard`** (default on a TTY) — full-screen live dashboard at 10 Hz, with the BEAM pids running right now visible in the "Running" column and completions streaming into the right-hand side.
- **`--stream`** — one line per test as it finishes. Colored. TAP-compatible with `--tap`.
- **`--quiet`** — only the final summary.

The dashboard is one of the more visible payoffs of the actor model: every test is a BEAM process, and the dashboard shows those pids as they enter and leave the "running" set. You get to *see* the parallelism.

Structured output for CI

Three flags emit machine-readable results:

```
--tap                # TAP 13 to stdout
--junit PATH         # JUnit XML to a file
--json PATH          # JSON to a file
```

These are designed to slot into any CI system without special plugins. TAP in particular is the universal test protocol — pipe it into any TAP consumer and you're done.

Filters and reproducibility

```
--match PATTERN      # substring match on test name
--tag LABEL          # only tests with this tag
--seed N             # fix the scheduler seed for reproducible parallel order
```

The seed is *always* reported in the summary, so a flaky run can always be replayed by passing the seed back.

Coverage

```
--coverage               # enable token-level coverage accumulation
--coverage-threshold N   # custom floor (default 95%)
--lax-coverage           # disable the threshold gate
```

Coverage is computed by stamping every dispatched token's `{file, line, column}` position during traced dispatch. At the end of the run, the runner tokenizes every `lib/**/*.a4` that isn't a test or DSL internal, counts positions, and compares against what was visited. Below the threshold, the run exits non-zero.

`.test.a4` files and `lib/testing/*.a4` are explicitly excluded from the denominator: tests don't measure themselves, and the DSL is exercised by construction on every run.

Diagnostics when a test fails

When a test fails, the runner's diagnostic engine inspects the fail-time data stack and classifies the failure into one of six categories from the test-DSL plan: *extra atom*, *missing consumer*, *wrong order*, *wrong clause fired*, *silent type mismatch*, *suddenly deep*. A typical failure line looks like:

```
not ok <0.213.0> talk_demo/lists/broken cons chain  891us
  {error, {af_error, assert_eq_failed, "Expected 5 but got foo", ...}}
  extra_atom: extra atom "foo" on stack (fallthrough: no op bound
    to this name, or not yet defined in this scope). Check spelling
    and scope.
    at samples/talk_demo.test.a4:57:15 (token: assert-eq)
```

The engine is itself written mostly in ActorForth (skeletons in `lib/testing/diagnose.a4`, category detection currently Erlang-side for speed). Users can add custom categories without touching any Erlang.

One-time isolation: `dict-snapshot` / `dict-restore`

Every test runs in its own fresh continuation, so word definitions inside a test body don't leak to other tests. If you want to snapshot and restore within a test — say, to test that `forget` behaves correctly — the primitives `dict-snapshot` and `dict-restore` (from `af_test_prim`) give you per-test dictionary checkpointing.

Size — measured

After porting 14 test suites from Erlang EUnit to `.test.a4`, the average reduction is **41%** in line count, for equivalent behavioral coverage. The range is +8% to −75% depending on how much of the Erlang test was fixture boilerplate versus actual assertions. Full table in `docs/design/test-dsl-plan.md`.

The point

The language ships with inline assertions (two words) and a test DSL (ten more words, written in ActorForth). The DSL itself is a demonstration of the language — it's written in the same language it tests. When you want tests to do something the DSL doesn't do yet, you write the word; you don't wait for a release.

Chapter 9: Maps

Maps are dynamic key-value structures — the complement to product types. Where product types have fixed schemas defined at type-definition time, maps can hold any keys and values, added and removed at runtime.

Building Maps

`map-new` pushes an empty map. `map-put` adds entries:

```
ok: map-new 100 "Alice" map-put 200 "Bob" map-put
ok: map-size
ok: stack
0) 2 : Int
```

The stack order for `map-put` is: map (deepest), value, key (TOS). Think of it as: "starting with this map, store this value at this key."

Retrieving Values

```
ok: map-new 42 "answer" map-put
ok: "answer" map-get
ok: stack
0) 42 : Int
```

`map-get` takes a key and a map, and pushes the value. If the key doesn't exist, you get a clear error:

```
Error: map_key_not_found at stdin:1:10
  Key "missing" not found in map
  Stack(2): "missing":String <Map>:Map
```

Other Map Operations

```
ok: map-new 1 "a" map-put 2 "b" map-put

ok: dup "a" map-has?       # -> True
ok: drop dup "c" map-has?  # -> False
ok: drop dup map-keys      # -> List of keys
ok: drop map-values        # -> List of values
```

`map-delete` removes a key. `map-size` returns the count of entries.

Maps with Product Types

Maps and product types work well together. A product type can have a Map field:

```
type Config
    name String
    settings Map .

"myapp" map-new 8080 "port" map-put config
```

This creates a Config with a fixed `name` field and a flexible `settings` map.

Chapter 10: The OTP Bridge

ActorForth actors can participate in Erlang/OTP applications today — without BEAM compilation. The `af_server` module is a gen_server that wraps an ActorForth interpreter, making ActorForth actors supervisable, callable from Erlang or Elixir, and fully integrated with OTP.

How It Works

Write your actor logic in a `.a4` file:

```
# bridge_counter.a4
type Counter
    value Int .

: increment Counter -> Counter ;
    dup value 1 + value!.

: count Counter -> Counter Int ;
    dup value.

: add Counter Int -> Counter ;
    swap dup value rot + value!.

0 counter
```

The last line leaves a Counter instance on the stack. `af_server` loads this file and uses that instance as the server's state.

Calling from Erlang

```
{ok, Pid} = af_server:start_link("samples/bridge_counter.a4"),

%% Cast (async, no return):
af_server:cast(Pid, "increment", []),

%% Call (sync, returns values):
{ok, [1]} = af_server:call(Pid, "count", []),

%% Call with arguments:
af_server:call(Pid, "add", [10]),
{ok, [11]} = af_server:call(Pid, "count", []),

%% Stop:
af_server:stop(Pid).
```

Term Conversion

At the bridge boundary, Erlang terms are automatically converted to ActorForth stack items and back:

Erlang	ActorForth
`42`	`{Int, 42}`
`<<"hello">>`	`{String, <<"hello">>}`
`true` / `false`	`{Bool, true/false}`
`foo` (atom)	`{Atom, "foo"}`
`[1, 2, 3]`	`{List, [{Int,1}, {Int,2}, {Int,3}]}`
`#{<<"k">> => 1}`	`{Map, ...}`

Product type instances convert to Erlang maps with a `type` key: `#{type => 'Counter', value => 0}`.

Supervision

Because `af_server` is a gen_server, it slots into supervision trees:

```
ChildSpec = #{
    id => my_counter,
    start => {af_server, start_link, ["counter.a4"]},
    restart => permanent,
    type => worker
},
```

If the ActorForth actor crashes, the supervisor restarts it — loading the `.a4` file fresh and starting from the initial state. This is standard OTP behavior, no special handling needed.

Raw Evaluation

For more flexible interaction, `af_server:eval/2` interprets a raw ActorForth line against the server's current state:

```
af_server:eval(Pid, "dup value 5 + value!").
```

This is useful for debugging, REPL-like interaction, or dynamic behavior.

Chapter 11: The Interpreter Is the Language

We've saved the deepest idea for near the end, because it requires everything before it to make sense.

Traditional language implementations have separate phases: parsing, analysis, compilation, execution. ActorForth has **one phase**: interpretation. Everything — including compilation — happens through the same dispatch mechanism.

The Five Steps, Always

Every token, every time:

1. Look in the TOS type's dictionary
2. Check the TOS type's handler
3. Look in the Any dictionary
4. Try literal handlers
5. Push as Atom

That's it. When you're "just running code," TOS is Int or Bool or whatever, and tokens resolve to arithmetic or stack operations. When you're "compiling a word," TOS is CodeCompile, and tokens resolve to "record this operation for later." The interpreter doesn't know the difference. It doesn't need to.

Creating New Syntax

This is where the idea becomes truly powerful. The compiler isn't special — it's just four types with dictionaries. **You can create your own types with their own dictionaries.** Any type can intercept tokens and give them meaning.

The product type system works this way. When you type `type`, it pushes a `TypeDefinition` onto the stack. Now every subsequent token is interpreted in the context of defining a type — the type name, then alternating field names and field types — until `.` finishes the definition. Same mechanism. Same interpreter. Different types on the stack.

The actor send protocol works the same way. `<<` pushes an `ActorSend` onto the stack. Now tokens are classified as

local or remote and dispatched accordingly. `>>` finishes the send block. No special syntax — just types redirecting what tokens mean.

You could define a type whose dictionary contains words for describing state machines. Or regular expressions. Or SQL queries. Or BEAM bytecode instructions. Each domain gets its own syntax, built from the same primitive: types with dictionaries.

This is homoiconicity in a new form. The language doesn't distinguish between code and data because the mechanism for processing both is the same: look up the token in the current context's dictionary. Code is data that happens to be in a dictionary. Data is tokens that didn't match any dictionary.

Late Binding and Recursion

When a word is compiled, its body isn't resolved to fixed function pointers. Each operation in the body is a thunk — a small piece of code that says "when executed, look up this token through the interpreter." This means:

- **Recursion works.** A word can call itself because the lookup happens at runtime, when the word already exists in the dictionary.
- **Forward references work.** A word can call a word that hasn't been defined yet. As long as it exists by the time the call happens, it's fine.
- **Redefinition works.** If you redefine a word, existing words that call it pick up the new definition automatically.

Chapter 12: Loading Files

ActorForth programs can be split across multiple files using the `load` word.

Libraries

Create a library file with reusable definitions:

```
# lib_math.a4
: square Int -> Int ; dup * .
: double Int -> Int ; 2 * .
: cube   Int -> Int ; dup dup * * .
```

Load it from another file or the REPL:

```
ok: "lib_math.a4" load
ok: 5 square
ok: stack
0) 25 : Int
```

The `load` word takes a String path, reads the file, parses it, and interprets it into the current continuation. All word definitions from the loaded file become available immediately. Relative paths are resolved from the loading file's directory.

Building Programs from Parts

```
# main.a4
"lib_math.a4" load
"counter.a4" load

5 square double print    # uses both libraries
```

The `load` word preserves the current stack state — it only adds definitions, it doesn't clear anything.

Chapter 13: Erlang FFI

ActorForth runs on the BEAM VM, and sometimes you need to call existing Erlang functions directly. The FFI (Foreign Function Interface) provides this bridge.

Zero-Argument Calls

```
ok: node erlang erlang-apply0
ok: stack
0) nonode@nohost : Atom
```

Push the function name, then the module name, then `erlang-apply0`. The result is automatically converted to an ActorForth stack item.

Calls with Arguments

For functions that take arguments, build an argument list first:

```
ok: [ -42 ] abs erlang erlang-apply
ok: stack
0) 42 : Int
```

Arguments go in a List, then function name, module name, then `erlang-apply`.

```
ok: [ 20 10 ] max erlang erlang-apply
ok: stack
0) 20 : Int
```

Type Conversion

The FFI automatically converts between Erlang and ActorForth types: - Erlang integers ↔ `{Int, N}` - Erlang binaries ↔ `{String, B}` - Erlang atoms ↔ `{Atom, S}` - Erlang booleans ↔ `{Bool, V}` - Erlang lists ↔ `{List, Items}` - Erlang maps ↔ `{Map, M}`

When to Use FFI

The FFI is for calling existing Erlang/OTP libraries — file I/O, networking, crypto, date/time, etc. For new logic, write it in ActorForth. The FFI is a bridge, not a crutch.

Chapter 14: Putting It All Together

Let's trace through a complete example that uses types, words, lists, pattern matching, and actors.

A Stateful Counter Actor

From `samples/counter_actor.a4`:

```
type Counter
    value Int .

: increment Counter -> Counter ;
    value 1 + value!.

: decrement Counter -> Counter ;
    value 1 - value!.

: count Counter -> Counter Int ;
    value.

# Create a counter starting at 0 and make it an actor
0 counter server

# Increment three times (cast - async)
<< increment >>
<< increment >>
<< increment >>

# Decrement once
<< decrement >>

# Get the count (call - sync) and verify
<< count >> 2 assert-eq

# Increment two more
<< increment >>
<< increment >>
<< count >> 4 assert-eq

# Clean up
```

```
<< stop >> drop
```

Every line is either a definition or a test. The assertions at the bottom prove the actor behaves correctly. Run the file and silence means it works.

A Temperature Converter

```
: f-to-c Int -> Int ; 32 - 5 * 9 / .
: c-to-f Int -> Int ; 9 * 5 / 32 + .

212 f-to-c 100 assert-eq
0 c-to-f 32 assert-eq
```

Small words, clear intent, self-testing.

Fibonacci with Pattern Matching

From `samples/fib.a4`:

```
: fib Int -> Int ;
    : 0 -> 0 ;
    : 1 -> 1 ;
    : Int -> Int ;
        dup
        1 -
        fib
        swap
        2 -
        fib
        +.

0 fib 0 assert-eq
1 fib 1 assert-eq
6 fib 8 assert-eq
10 fib 55 assert-eq
```

No branching. No if/else. The dispatch mechanism selects the right clause based on the value on the stack.

An Event Statistics Actor

```
type Stats
    hits Int
    errors Int .

: hit Stats -> Stats ;
    hits 1 + hits!.

: err Stats -> Stats ;
    errors 1 + errors!.

: report Stats -> Stats Int Int ;
    hits swap errors rot.

0 0 stats server

<< hit >>
<< hit >>
<< hit >>
<< err >>
<< report >>
# Stack now has: 3 (hits), 1 (errors), Actor
```

Two counters in one actor. Each operation touches only the field it cares about. The `report` call returns both values synchronously.

Chapter 15: Compiling Words

ActorForth includes two compilation strategies that go beyond the interpreter's late-binding dispatch.

Closure-Based Compilation

`af_compile:compile_word/4` takes a word's name, input/output signatures, and body operations, and pre-resolves all body tokens at compile time. Known primitives (`dup` , `drop` , `swap` , `+` , `-` , `*`) get replaced with optimized inline funs that skip the interpreter entirely:

```
%% Instead of ETS lookup + dispatch for every "dup":
fun(S) -> [hd(S) | S] end
```

The result is an `#operation{}` with `source = compiled` that can replace the interpreted version.

BEAM Module Generation

`af_compile:compile_module/2` goes further — it generates a real BEAM module via `compile:forms/2` . Each word becomes an exported function:

```
%% From ActorForth: : double Int -> Int ; dup + .
%% Generated:
double(Stack0) ->
    af_compile:apply_impl("dup", af_compile:apply_impl("+", Stack0)).
```

The module is loaded dynamically with `code:load_binary/3` . Words in the module are callable as regular Erlang functions.

Tail Call Optimization

When a word's body ends with a call to any compiled word (including self-calls), the compiler detects this and restructures the closure so the call is in Erlang tail position. This means tail-recursive ActorForth words — and mutually-recursive word chains — use constant stack space:

```
: countdown Int -> ;
    : 0 -> ; drop
    : Int -> ; 1 - countdown .

1000 countdown    # Uses O(1) stack, not O(n)
```

The mechanism: the word trace frame is popped *before* the tail call, putting it in true Erlang tail position. The BEAM's native TCO handles the rest.

Chapter 16: Supervised Actors

While `server` spawns a raw Erlang process, `supervised-server` creates an actor under OTP supervision — giving you automatic restart on failure.

Basic Usage

```
type Counter value Int .
: increment Counter -> Counter ; value 1 + value! .
: count Counter -> Counter Int ; value .

0 counter supervised-server
```

This spawns the actor under a `simple_one_for_one` supervisor. The actor is implemented as a `gen_server` that handles `cast` (async) and `call` (sync) messages through the same word dispatch mechanism.

How It Works

- `supervised-server` creates a `gen_server` worker via `af_actor_worker`
- The supervisor (`af_actor_sup`) uses `transient` restart strategy — actors restart on abnormal termination but not on normal `stop`
- `<< ... >>` send blocks work identically for both `server` and `supervised-server` actors
- The `supervised` flag in the Actor map routes messages through `gen_server:call/cast` instead of raw `!`

Differences from `server`

Feature	`server`	`supervised-server`
Process type	`spawn_link`	`gen_server` under supervisor
Crash behavior	Crashes caller	Restarts automatically
Message passing	Raw `!` and `receive`	`gen_server:cast/call`
Stop behavior	Process exits	`{stop, normal, State}`

Chapter 17: HOS — Structured State Machines

Actors are general; they accept any sequence of events and let the programmer decide what each one means. That generality is exactly right for most software. It is exactly wrong for software where the cost of an unexpected event is measured in lives or money.

For that narrower problem, ActorForth ships an optional layer on top of the actor model: HOS, Margaret Hamilton's design discipline for hierarchies of state machines with axiomatic correctness checks. HOS came out of Apollo. Its value proposition is sharp: explicit, declared hierarchical state machines where safety properties emerge from structure rather than from programmer vigilance. Lifts, payment terminals, signalling, medical devices, protocol state. Not CRUD, not aggregation, not games, not most web backends. If your problem is a state machine that has to be right, HOS earns its keep. If it is not, you can skip this chapter with no loss.

A First System

The smallest interesting state machine is a turnstile. Two states (Locked, Unlocked), two events (coin inserted, person pushes the bar):

```
system Turnstile
  state TurnstileState
  on coin -> ;
  on push -> ;
  transitions
    Locked   -> Unlocked coin
    Unlocked -> Locked   push
end
```

Read this top to bottom. The system is named `Turnstile`. It is **stateful** (it carries a `TurnstileState`, an atom that names the current state). It declares two events it knows about: `coin` and `push`. The transitions table is the law: from `Locked`, a `coin` event moves to `Unlocked`; from `Unlocked`, a `push` event moves to `Locked`.

Three things follow from this declaration that you should notice.

First, what is **not** there. There is no body code on either handler. The `on coin -> ;` and `on push -> ;` declarations have empty bodies. On a stateful HOS system, that is the rule, not an oversight. The transitions table decides what happens; the handler bodies cannot.

Second, what is **silently safe**. A `push` event arriving in the `Locked` state is not in the transitions table for that state. The runtime input-rejects it. No state change. No exception. No error buried in a handler that forgot to check. The structural shape of the table is the safety property.

Third, what the checker has already done before this code ran. It verified that every state name on the right-hand side of a transition appears as a `from` somewhere (so the state graph is connected from the initial state), that every trigger event was declared in an `on` line, and that no body referenced a name outside this system's scope.

What the Checker Enforces

HOS is a discipline; the checker is what makes it stick. There are four rules worth understanding before you write your second system.

Axiom 1: immediate-only control. A system may reference its own direct children, its own parent, its own state names, its own handler events, and a4 builtins. It may not reach into siblings, cousins, or arbitrary helper functions defined elsewhere. If you accidentally write a handler body that calls a sibling system, the checker rejects the declaration at definition time. The structural payoff: you can read one system in isolation and know its full surface. There are no hidden dependencies.

Axiom 5: input rejection. The transitions table is exhaustive for the states it declares. An event arriving in a state that has no row for that trigger is rejected (no state change). The handler body cannot sneak in a state change that is not in the table. The payoff: every state change is visible in one place. Reorderings, renames, and missing edges show up as

table rows, not as scattered conditionals.

One event fires one transition. There is no cascade. A handler cannot run, look at conditions, and then fire another event in the same step. If a multi-step sequence is needed (open the door, then start the motor, then announce the floor), each step is a separate event arriving at the system. This is structural, not stylistic. It makes interleaving and concurrency tractable to reason about.

Stateful handler bodies are empty. If a system declares a state type, its `on X -> ;` handler bodies must be empty. The declaration documents which events the system knows about; the transitions table decides what they do. (Stateless systems, declared with `state None`, are routers, not state machines, and may have body code that dispatches to children.)

Effects Live in Transitions

Real systems do more than change their own state; they tell other systems to do things. In HOS, those "tell other systems" calls are declared as part of the transition that triggers them, not buried in handler body code. The shape:

```
transitions
    Locked -> Unlocked coin
    Unlocked -> Locked push : LightStrip flash-green
```

The trailing `: LightStrip flash-green` declares: when this transition fires, also send the `flash-green` event to the `LightStrip` child. That subsystem call is now visible in the table. If somebody later changes the target system or the event name, the diff is one line in this file, not one line buried in body code somewhere.

For physical actuation that takes time (a motor that needs three seconds to reach the next floor), the timer primitive lives here too:

```
transitions
    Idle -> Moving go : after 3000 arrived
```

After this transition fires, the runtime schedules an `arrived` event to itself in 3000 milliseconds. The mailbox stays open during the wait, so events arriving in the meantime are handled normally (and may even pre-empt the timer, if their transitions are declared from the live state). This is the structural replacement for "hold a lock and sleep" or "run a worker thread."

The Hierarchy

Systems compose into trees. A parent declares its children by name in a `children` section; the children declare their parent. The Axiom 1 scope check uses both directions: a parent may reference its declared children, a child may reference its declared parent, but neither may reach across the tree to a sibling.

Child-to-parent signals (the child has finished its work and the parent should advance) are modelled as effects targeting the parent by name. There is no special "send to parent" primitive. The parent is declared, named, and addressed exactly like any other subsystem. This is the same idea that makes Erlang supervision trees work: the hierarchy is the data structure that owns the policy.

The full elevator example, which uses a four-level hierarchy (Building → Floor → Elevator → Motor) to coordinate physical actuation, child-to-parent signals, and structural prevention of classic textbook bugs, lives in `docs/hos/elevator.md`. Read that chapter when you are ready to see HOS work at scale.

When NOT to Reach for HOS

The discipline costs attention. Every system declaration is a type-checked artifact; every event needs an `on` line; every state change needs a row in the table. For state-machine code that has to be right, that cost is the point. For code that does not have to be right in that way (an aggregation pipeline, a UI form, a recommendation model, a CRUD endpoint), the cost is friction without payoff.

A useful rule of thumb: if you can describe your problem as a list of states and a list of allowed transitions between them, and getting the list wrong has consequences you would not want to debug at 3am, HOS is probably the right tool. If you are reaching for HOS to model "the shape of my data flowing through stages," you probably want the actor model directly, or even just plain words and product types.

Beyond v1

The current HOS surface ships the four checks above plus the timer primitive and the hierarchy. Several pieces are scoped for future work: multi-instance children (a building with N elevators where each is its own actor), preconditions on transitions (only allow `coin` when the cash drawer is not full), and structured data on events (today an event is just

an atom with no payload). The `docs/hos/elevator.md` chapter discusses the v2 boundaries in more detail.

Chapter 18: Compile-Time Type Checking

ActorForth checks word definitions at compile time by inferring the stack effect of each body token and comparing the result against the declared output signature.

How It Works

When you define a word:

```
: double Int -> Int ; dup + .
```

The compiler: 1. Starts with the declared input stack: `[Int]` 2. Looks up `dup` — sig is `Any -> Any Any` — but since the input is `Int`, resolves `Any` to `Int`: stack becomes `[Int, Int]` 3. Looks up `+` — sig is `Int Int -> Int` — consumes two Ints, produces one: stack becomes `[Int]` 4. Compares `[Int]` against declared output `[Int]` — match!

If there's a mismatch, the compiler emits a warning:

```
: bad Int -> Int ; dup .
Warning: type mismatch in word 'bad'
  declared output: ['Int']
  inferred output: ['Int','Int']
```

Type Variable Resolution

The type checker resolves `Any` in operation signatures to concrete types from the actual stack. This means `dup` applied to `Bool` correctly infers `[Bool, Bool]`, not `[Any, Any]`. The `Any` type acts as a type variable that gets unified during inference.

Enforcement

As of v2.3, type checking is **enforced** — verified type mismatches are compile-time errors:

```
: bad Int -> Bool ; dup + .
** error: Type mismatch in word 'bad': declared output [Bool] but inferred [Int]
```

When inference is incomplete (product type operations, unknown words), the compiler emits a **warning** instead of an error, since the type checker can't fully resolve the stack:

```
: increment Counter -> Counter ; value 1 + value! .
Warning: type check for 'increment' incomplete
  declared output: ['Counter']
  inferred output: ['Atom','Int']
```

Limitations

- Recursive calls and unknown words are treated as pushing `Atom` — full inference for user-defined words requires iterative analysis
- Pattern matching sub-clauses are checked independently per clause
- Product type operations (getters/setters) aren't fully resolved during inference, resulting in warnings rather than errors

Chapter 19: The BEAM Assembler

ActorForth can compile functions directly to BEAM bytecode using the `BeamModule` and `BeamFunction` types. This is the first step toward self-hosting.

Building a BEAM Module

```
# Create a module
my_math beam-module

# Define: double(X) -> X + X
double 1 int beam-fun
    1 int beam-arg        # push Arg1
```

```
    1 int beam-arg        # push Arg1 again
    + 2 int beam-op       # apply + to the two args
beam-return

# Define: square(X) -> X * X
square 1 int beam-fun
    1 int beam-arg
    1 int beam-arg
    * 2 int beam-op
beam-return

# Compile and load the module
beam-compile
# Stack: [Atom("my_math")]
```

After `beam-compile`, the module is loaded and callable from Erlang:

```
my_math:double(5).   %% => 10
my_math:square(7).   %% => 49
```

Available Words

Word	Stack Effect	Description
`beam-module`	`( Atom -- BeamModule )`	Start new module
`beam-fun`	`( BeamModule Atom Int -- BeamFunction )`	Start function (name, arity)
`beam-arg`	`( BeamFunction Int -- BeamFunction )`	Reference argument N (1-based)
`beam-int`	`( BeamFunction Int -- BeamFunction )`	Integer literal
`beam-atom`	`( BeamFunction Atom -- BeamFunction )`	Atom literal
`beam-op`	`( BeamFunction Atom Int -- BeamFunction )`	Apply operator (+, -, *, etc.)
`beam-call`	`( BeamFunction Atom Atom Int -- BeamFunction )`	Remote call Mod:Fun/Arity
`beam-return`	`( BeamFunction -- BeamModule )`	Finish function
`beam-compile`	`( BeamModule -- Atom )`	Compile and load module

How It Works

The assembler builds Erlang abstract forms — the same format `compile:forms/2` expects. Each `beam-*` word adds to the form tree. `beam-compile` hands the forms to the Erlang compiler, which generates optimized BEAM bytecode and loads it into the VM.

This demonstrates the core principle: **the types on the stack determine what tokens mean.** When a `BeamFunction` is on the stack, `beam-arg` adds an argument reference. When a `BeamModule` is on the stack, `beam-compile` triggers compilation. The outer interpreter doesn't change — it's the same five-step dispatch driving everything.

Chapter 20: Introspection with `see`

The `see` word lets you inspect any word's definition — its type signature, body, and which type dictionary it lives in.

Viewing Built-in Words

```
ok: dup see
  : dup Any -> Any Any ;  [built-in]
    in type: 'Any'
```

Viewing Compiled Words

```
ok: : double Int -> Int ; dup + .
ok: double see
  : double Int -> Int ; dup + .
    in type: 'Int'
```

For compiled words, `see` shows the body tokens — the same words you wrote in the definition.

Multiple Implementations

Some words have different implementations across type dictionaries. `see` shows all of them:

```
ok: int see
  : int Atom -> Int ;  [built-in]
    in type: 'Any'
  : int Atom(0) -> Int ;  [built-in]
    in type: 'Any'
```

The `int` constructor has two signatures: a general one for any Atom, and a specialized one matching the literal `0`. When you type `0 int`, the value-constrained signature matches first.

Native Wrappers

After compiling a word to native BEAM (see next chapter), `see` shows the native module:

```
ok: double see
  : double Int -> Int ;  [native: af_ctb_wrap]
    in type: 'Int'
```

Chapter 21: Compiling to Native BEAM

ActorForth can compile word definitions into native BEAM modules — producing real Erlang-callable functions with full performance. This bridges the gap between the interpreted stack language and the BEAM runtime.

The Workflow

Define a word, compile it, use it — all from the REPL:

```
ok: : double Int -> Int ; dup + .
ok: 5 int double
ok: stack
0) 10 : Int

# Compile to a BEAM module
ok: double af_math compile-to-beam
ok: stack
0) af_math : Atom

# The word still works — now via native code
ok: 7 int double
ok: stack
0) 14 : Int
1) af_math : Atom

# Also callable from Erlang:
# af_math:double(5) => 10
```

What Happens

`compile-to-beam` does three things:

1. **Translates** the word body into Erlang abstract forms. Stack operations (`dup`, `swap`, `drop`) become expression manipulations. Arithmetic (`+`, `-`, `*`, `/`) becomes Erlang operators. Integer literals become constants.
2. **Compiles** the abstract forms into a BEAM module using `compile:forms/2`, then loads it with `code:load_binary/3`.
3. **Replaces** the interpreted word with a **transparent native wrapper**. The wrapper unwraps `{Type, Value}` tagged items from the ActorForth stack, calls the native BEAM function, and re-wraps the results. ActorForth code using the word doesn't notice any change.

Batch Compilation with `compile-all`

When you have multiple words, `compile-all` compiles them all into a single module:

```
ok: : double Int -> Int ; dup + .
ok: : square Int -> Int ; dup * .
ok: : inc Int -> Int ; 1 + .
ok: af_math compile-all
ok: stack
0) af_math : Atom

# All three functions are now native
ok: 5 int double
ok: stack
0) 10 : Int
1) af_math : Atom
```

From Erlang: `af_math:double(5)`, `af_math:square(7)`, `af_math:inc(3)`.

Saving Modules to Disk

`save-module` writes the compiled BEAM binary to a directory:

```
ok: : double Int -> Int ; dup + .
ok: double af_math compile-to-beam
ok: "/path/to/ebin" save-module
```

The resulting `af_math.beam` file can be added to any Erlang/Elixir project's code path.

Building Standalone Applications

`build-escript` creates a self-contained executable from a compiled module:

```
ok: : double Int -> Int ; dup + .
ok: double af_math compile-to-beam
ok: double "./my_app" build-escript
```

The escript bundles the compiled module with a generated `main/1` entry point. Run it from the command line:

```
$ ./my_app
# Calls af_math:double() or the specified entry point
```

Complete REPL-to-Executable Example

```
# 1. Define your words
ok: : fib Int -> Int ;
    : 0 -> 0 ;
    : 1 -> 1 ;
    : Int -> Int ; dup 1 - fib swap 2 - fib + .

# 2. Test interactively
ok: 10 int fib
ok: stack
0) 55 : Int

# 3. Compile to native BEAM
ok: fib af_fib compile-to-beam

# 4. Save the module
ok: "/my/project/ebin" save-module

# 5. Or build a standalone executable
ok: fib "./fib_app" build-escript
```

Inter-Word Compilation

When `compile-all` compiles multiple words into the same module, calls between words become direct BEAM function calls — no interpreter dispatch overhead:

```
ok: : double Int -> Int ; dup + .
```

```
ok: : quadruple Int -> Int ; double double .
ok: af_math compile-all
# quadruple calls double directly as a native BEAM call
```

OTP Application Packaging

`build-release` creates a proper OTP application directory structure:

```
ok: : double Int -> Int ; dup + .
ok: double af_math compile-to-beam
ok: af_math "1.0.0" "/my/releases" build-release
# Creates /my/releases/af_math-1.0.0/ebin/ with .app and .beam files
```

OTP Behaviour Generation

`gen-server-module` compiles an ActorForth product type and its words into a proper gen_server module:

```
ok: type Counter value Int .
ok: : increment Counter -> Counter ; value 1 + value! .
ok: : count Counter -> Counter Int ; value .
ok: af_counter Counter gen-server-module

# Now af_counter is a gen_server module:
# gen_server:start_link(af_counter, InitState, [])
# gen_server:cast(Pid, increment)
# gen_server:call(Pid, count) => {ok, 3}
```

The generated module can be placed in any OTP supervision tree.

Limitations

- Words calling other user-defined words within the same module compile to direct native calls. Cross-module calls to already-compiled native words are also resolved.
- Words using operations that aren't known primitives fall back to runtime dispatch — the ActorForth runtime must be present.
- Zero-argument words (like `: answer -> Int ; 42 .`) auto-execute when their name appears, so you can't push their name as an Atom for `compile-to-beam`. Use the Erlang API (`af_word_compiler:compile_words_to_module/2`) for those.

Chapter 22: Full BEAM Interoperability

ActorForth is a first-class BEAM language. Here's the full interoperability picture.

Calling Erlang/Elixir from ActorForth

The `erlang-call` word provides natural FFI — push arguments, then module, function, and arity:

```
ok: -5 int erlang abs 1 int erlang-call
ok: stack
0) 5 : Int

ok: 3 int 7 int erlang max 2 int erlang-call
ok: stack
0) 7 : Int
```

For Elixir modules (they're atoms like `'Elixir.String'`):

```
ok: "hello" Elixir.String upcase 1 int erlang-call
```

Calling ActorForth from Erlang/Elixir

Compiled words are standard BEAM functions:

```
%% From Erlang
af_math:double(5).        %% => 10
af_math:square(7).        %% => 49
```

```
%% From Elixir
AfMath.double(5)           # => 10
```

OTP Integration

Generated gen_server modules work in any supervision tree:

```
%% Start the ActorForth-compiled gen_server
{ok, Pid} = gen_server:start_link(af_counter, InitState, []),
gen_server:cast(Pid, increment),
{ok, Count} = gen_server:call(Pid, count).
```

Build Integration

The rebar3 plugin compiles `.a4` files automatically:

```
%% rebar.config
{plugins, [rebar3_actorforth]}.
```

Or compile programmatically:

```
af_compile_file:compile("src/math.a4").
af_compile_file:compile_to_dir("src/math.a4", "ebin/").
```

Type Bridging

`af_term` handles bidirectional conversion at all boundaries:

Erlang	ActorForth
`42`	`{Int, 42}`
`3.14`	`{Float, 3.14}`
`true`	`{Bool, true}`
`<<"hello">>`	`{String, <<"hello">>}`
`foo`	`{Atom, "foo"}`
`{ok, 42}`	`{Tuple, {ok, 42}}`
`[1, 2]`	`{List, [{Int,1}, {Int,2}]}`
`#{k => v}`	`{Map, ...}`

Chapter 23: Python Interop

ActorForth can call Python code directly via the `erlang_python` library, which embeds Python in the BEAM VM. This opens access to Python's AI/ML ecosystem from ActorForth programs.

Getting Started

```
ok: py-start
ok: "2 + 2" py-eval
ok: stack
0) 4 : Int
```

`py-start` ensures the Python runtime is running. `py-eval` evaluates a Python expression and pushes the result.

Calling Python Functions

```
ok: 16 math sqrt 1 py-call
ok: stack
0) 4.0 : Float
```

`py-call` takes N arguments from the stack, plus the module name and function name. The format is: `arg1 ... argN module function arity py-call`. For zero-argument functions, use `py-call0`:

```
ok: random random py-call0
ok: stack
0) 0.7234... : Float
```

Custom Python Modules

Add a directory to Python's import path with `py-import`, then call functions from your modules:

```
ok: "samples/python" py-import
ok: "hello world" text_tools word_count 1 py-call
ok: stack
0) 2 : Int
```

Executing Python Statements

```
ok: "import sys" py-exec
```

`py-exec` runs Python statements without returning a value. Note: functions defined via `py-exec` are not visible to `py-eval` due to scope isolation — use module-level state via imported modules instead.

Virtual Environments

```
ok: ".venv" py-venv
```

`py-venv` activates a Python virtual environment, making its packages available.

Registering ActorForth Words for Python

Words compiled to native BEAM can be registered as Python-callable:

```
ok: : double Int -> Int ; dup + .
ok: double af_math compile-to-beam
ok: double py-register
```

Now Python code can call `erlang.double(21)` and get `42`.

AI/LLM Integration

The `samples/python/llm_client.py` module provides OpenAI/Anthropic access:

```
ok: py-start
ok: "samples/python" py-import
ok: "What is Erlang?" llm_client chat 1 py-call
ok: print
Erlang is a functional programming language...
```

The REPL automatically loads `.env` on startup, so `OPENAI_API_KEY` set in `.env` is available to Python modules via `os.environ`.

Type Conversion

ActorForth stack items are automatically converted when passed to/from Python:

ActorForth	Python
`{Int, 42}`	`42`
`{Float, 3.14}`	`3.14`
`{Bool, true}`	`True`
`{String, "hello"}`	`"hello"` (str)
`{Atom, "foo"}`	`"foo"` (str)
`{List, Items}`	`list`

Chapter 24: Inter-Module Imports

The `import` word compiles a `.a4` file to a native BEAM module and makes its words available:

```
ok: "lib_math.a4" import
ok: stack
0) lib_math : Atom

ok: 5 double    # uses the imported word
ok: stack
0) 10 : Int
```

How It Works

`import` calls `af_compile_file:compile/1` internally: 1. Reads and parses the `.a4` file 2. Interprets it to discover word definitions 3. Compiles all defined words to a BEAM module 4. Loads the module and registers native wrappers in the type registry

The imported words become native BEAM functions — faster than interpreted words and callable from Erlang.

Import vs Load

Feature	`load`	`import`
Execution	Interpreted	Compiled to BEAM
Performance	Late-binding dispatch	Direct native calls
Words available	In current continuation	In type registry as native
Erlang-callable	No	Yes (`module:function(args)`)
Return value	Nothing	Module name as Atom

Use `load` for scripts that need the current stack state. Use `import` for libraries of reusable words.

Path Resolution

Relative paths are resolved from the importing file's directory, or the current working directory if called from the REPL.

Environment Configuration

The REPL loads `.env` on startup. Create a `.env` file in your project root:

```
OPENAI_API_KEY=sk-xxx
DATABASE_URL=postgres://...
```

Lines starting with `#` are comments. Values can be quoted with `"` or `'`. The variables are set via `os:putenv/2` and available to both Erlang and Python code.

Chapter 25: The Road Ahead

ActorForth now has the full compilation pipeline, OTP integration, and BEAM interoperability needed to be a practical BEAM language. The remaining steps:

- **Core Erlang target**: Generate Core Erlang instead of abstract forms for more control over optimization.
- **Self-hosting**: Write the ActorForth compiler, type system, and BEAM assembler in ActorForth itself. The TOS-driven dictionary mechanism makes this possible — a compiler is just types for emitting code, using the same interpreter that runs user programs.
- **Mix integration**: A Mix task for Elixir projects using ActorForth modules.
- **Protocol support**: Let ActorForth types implement Elixir protocols and OTP behaviours beyond gen_server.

Chapter 26: ActorForth and LLMs

A note on why the DSL-first model turns out to be an unusually good fit for generating code with a language model — and where the limits are.

What LLMs are good at

Large language models are exceptional at two things that matter here:

1. **Writing small, self-contained pieces of code** from a natural-language description. Give an LLM "a function that folds a list and picks the maximum" and it produces that function across any language it has seen.
2. **Imitating a vocabulary.** Show an LLM a handful of examples of how a DSL is used, and it will produce more of them — often with eerie accuracy about the conventions.

They're noticeably worse at:

- Keeping a large program consistent over many files.
- Holding invariants in mind across refactors.
- Debugging when the mental model and the actual behavior diverge.

Why a4 fits the first two strengths

The canonical ActorForth workflow is: **build a DSL for your problem, then solve the problem in that DSL.** Both halves of this play to LLM strengths.

The DSL layer is small, self-contained code. New words are a few lines each. Each word fits in a token window. Each has a visible stack signature. An LLM producing a new word doesn't need to understand anything outside the signature it was given.

The domain layer is repeated vocabulary. Once five tests use `"name" test : body ;`, an LLM can produce a hundred more by pattern. The shape is so uniform it's almost a template.

The weakness of LLM code generation — long-range consistency — is mitigated two ways:

- Each word is **type-checked at definition time**, so inconsistencies surface immediately instead of compounding.
- The REPL lets you **run a suggested word in isolation** before committing. Feedback loop in seconds, not minutes.

What this looks like in practice

Given a problem like "write a DSL for financial trade lifecycle events," a typical ActorForth session with LLM assistance looks like:

1. You sketch the ontology in plain English. The LLM proposes a handful of words (`trade`, `match`, `settle`, `cancel`) with stack signatures.
2. You push back on signatures that don't make sense. A few rounds.
3. The LLM writes skeleton bodies. You run them at the REPL.
4. Type-check failures are short, specific, and local — the LLM corrects them.
5. Once the words work on one trade, you scale to a thousand via `map` or actors.

Compare to the same exercise in Python: the LLM writes a class hierarchy, you accept it, and two weeks later the class graph has seven intersecting concerns and the initial cost of understanding is no longer recoverable. The DSL approach front-loads clarity — the ontology *is* the code.

What it doesn't solve

LLMs are not magic. They still:

- Invent words that don't exist (the test DSL's `extra_atom` diagnostic catches this immediately).
- Get stack ordering wrong (the type checker catches this immediately).
- Hallucinate behavior for built-in primitives (a type mismatch catches this).

The common thread: **ActorForth's type system and immediate feedback turn these failure modes into quickly-correctable errors rather than silent long-range drift.** This is not because a4 is smarter; it is because a4 commits to smaller units of meaning.

A note on scope

The argument above is a design argument, not a marketing claim. ActorForth is not "the LLM language." It is a language whose existing design — driven by concerns that predate the current LLM moment by decades — happens to align well with how LLMs produce code. If LLM capabilities change, the language does not need to.

The canonical reference remains: *"The best a4 programs rarely use a4 primitives."* An LLM is just another way to get from the problem to the DSL. The language itself is concerned with the DSL and the problem, not the path between them.

Quick Reference

Stack Operations (Any dictionary)

Word	Effect	Description
`dup`	`( a -- a a )`	Duplicate top
`drop`	`( a -- )`	Discard top
`swap`	`( a b -- b a )`	Swap top two
`rot`	`( a b c -- b c a )`	Rotate third to top
`over`	`( a b -- a b a )`	Copy second to top
`2dup`	`( a b -- a b a b )`	Duplicate top two
`print`	`( a -- )`	Print and consume top
`stack`	`( -- )`	Display entire stack
`words`	`( -- )`	List all operations
`types`	`( -- )`	List all types
`see`	`( Atom -- )`	Show word definition and signatures
`load`	`( String -- )`	Load and interpret a .a4 file
`import`	`( String -- Atom )`	Compile .a4 to BEAM module, load words

Arithmetic (Int dictionary)

Word	Effect	Description
`+`	`( a b -- a+b )`	Addition
`-`	`( a b -- a-b )`	Subtraction (second minus top)
`*`	`( a b -- a*b )`	Multiplication
`/`	`( a b -- a/b )`	Integer division

Comparison (Any dictionary)

Word	Effect	Description
`==`	`( a b -- bool )`	Equal
`!=`	`( a b -- bool )`	Not equal
`<`	`( a b -- bool )`	Less than
`>`	`( a b -- bool )`	Greater than
`<=`	`( a b -- bool )`	Less or equal
`>=`	`( a b -- bool )`	Greater or equal

Boolean (Bool dictionary)

Word	Effect	Description
`not`	`( bool -- bool )`	Logical negation

Assertions (Any dictionary)

Word	Effect	Description
`assert`	`( Bool -- )`	Error with location if false

`assert-eq`	`( a b -- )`	Error with expected/actual if not equal

List Operations

Word	Dict	Effect	Description
`nil`	Any	`( -- List )`	Push empty list
`cons`	Any	`( a List -- List )`	Prepend item to list
`length`	List	`( List -- Int )`	Count elements
`head`	List	`( List -- a )`	First element
`tail`	List	`( List -- List )`	All but first

String Operations

Word	Dict	Effect	Description
`"hello"`	—	`( -- String )`	Quoted string literal
`string`	Any	`( Atom -- String )`	Explicit constructor
`concat`	String	`( Str Str -- Str )`	Concatenate (second + top)
`length`	String	`( Str -- Int )`	Byte length
`to-atom`	String	`( Str -- Atom )`	Convert to Atom
`to-int`	String	`( Str -- Int )`	Parse as integer
`to-string`	Int	`( Int -- Str )`	Convert to String

Map Operations

Word	Dict	Effect	Description
`map-new`	Any	`( -- Map )`	Push empty map
`map-put`	Any	`( Map val key -- Map )`	Add/update entry
`map-get`	Any	`( Map key -- val )`	Get value (errors if missing)
`map-delete`	Any	`( Map key -- Map )`	Remove entry
`map-has?`	Any	`( Map key -- Bool )`	Check key exists
`map-keys`	Map	`( Map -- List )`	All keys as list
`map-values`	Map	`( Map -- List )`	All values as list
`map-size`	Map	`( Map -- Int )`	Count of entries

Float Operations

Word	Dict	Effect	Description
`3.14`	—	`( -- Float )`	Float literal (auto-detected)
`float`	Any	`( Atom -- Float )`	Parse text as float
`to-float`	Int	`( Int -- Float )`	Convert Int to Float
`to-int`	Float	`( Float -- Int )`	Truncate Float to Int
`+` `-` `*` `/`	Float	`( Float Float -- Float )`	Arithmetic

Mixed Float/Int arithmetic is supported (e.g., `5.5 2 int +` → `7.5`).

Tuple Operations

Word	Dict	Effect	Description
`make-tuple`	Any	`( a... Int -- Tuple )`	Build tuple from N stack items
`to-tuple`	List	`( List -- Tuple )`	Convert list to tuple
`from-tuple`	Tuple	`( Tuple -- List )`	Convert tuple to list
`tuple-get`	Any	`( Int Tuple -- Any )`	Get element at 1-based index
`tuple-size`	Tuple	`( Tuple -- Int )`	Number of elements
`ok-tuple`	Any	`( Any -- Tuple )`	Create `{ok, Value}`
`error-tuple`	Any	`( Any -- Tuple )`	Create `{error, Reason}`
`is-ok`	Tuple	`( Tuple -- Bool )`	Check if starts with ok
`unwrap-ok`	Tuple	`( Tuple -- Any )`	Extract value from `{ok, V}`

Constructors (Any dictionary)

Word	Effect	Description
`int`	`( Atom -- Int )`	Parse text as integer
`bool`	`( Atom -- Bool )`	Parse "True"/"False" as boolean
`float`	`( Atom -- Float )`	Parse text as float
`42`	`( -- Int )`	Literal handler (no explicit constructor needed)
`3.14`	`( -- Float )`	Literal handler (no explicit constructor needed)
`True` / `False`	`( -- Bool )`	Literal handler (no explicit constructor needed)

Word Definition

```
: name  InputTypes -> OutputTypes ; body .
```

Pattern Matching Sub-Clauses

```
: name MasterSig ;
    : value-constrained-sig ;
    : value-constrained-sig ; body
    : general-sig ; body.
```

Product Type Definition

```
type TypeName
    field1 Type1
    field2 Type2 .
```

Auto-generates: constructor (`typename`), non-destructive getters (`field1` , `field2`), setters (`field1!` , `field2!`). Getters push the field value while leaving the instance on the stack.

Actor Operations

Word	Effect	Description
`server`	`( instance -- Actor )`	Spawn stateful actor from any typed instance
`supervised-server`	`( instance -- Actor )`	Spawn supervised actor (OTP gen_server)
`self`	`( -- Actor )`	Push current process reference
`<<`	Enter send mode	Start actor message block
`>>`	Exit send mode	End actor message block, push results

`stop`	(inside `<< >>`)	Terminate actor, push final state

OTP Bridge (Erlang API)

Function	Description
`af_server:start_link(Script)`	Start bridge server from .a4 file
`af_server:call(Pid, Word, Args)`	Sync call, returns `{ok, Results}`
`af_server:cast(Pid, Word, Args)`	Async cast, returns `ok`
`af_server:eval(Pid, Line)`	Evaluate raw ActorForth line
`af_server:stop(Pid)`	Stop the server

Erlang FFI (Any dictionary)

Word	Effect	Description
`erlang-apply0`	`( Atom Atom -- Any )`	Call Module:Function()
`erlang-apply`	`( Atom Atom List -- Any )`	Call Module:Function(Args...)
`erlang-call`	`( Any... Atom Atom Int -- Any )`	Call Module:Function with N stack args
`erlang-call0`	`( Atom Atom -- Any )`	Call Module:Function() (shorthand)
`erlang-new`	`( Atom -- Tuple )`	Call Module:new()

Native Compilation (Any dictionary)

Word	Effect	Description
`compile-to-beam`	`( Atom Atom -- Atom )`	Compile word to BEAM module
`compile-all`	`( Atom -- Atom )`	Compile all user words to one module
`save-module`	`( String Atom -- )`	Write .beam file to directory
`build-escript`	`( Atom String -- )`	Build standalone executable
`build-app`	`( String Atom -- )`	Generate .app file for module
`build-release`	`( String String Atom -- )`	Create OTP app directory structure
`gen-server-module`	`( Atom Atom -- Atom )`	Compile type to gen_server module

Python Interop (Any dictionary)

Word	Effect	Description
`py-start`	`( -- )`	Ensure Python runtime is started
`py-call`	`( a... Int Atom Atom -- Any )`	Call module.function with N args
`py-call0`	`( Atom Atom -- Any )`	Call module.function() (zero args)
`py-eval`	`( String -- Any )`	Evaluate Python expression
`py-exec`	`( String -- )`	Execute Python statements
`py-import`	`( String -- )`	Add directory to Python sys.path
`py-venv`	`( String -- )`	Activate Python virtual environment
`py-register`	`( Atom -- Atom )`	Register compiled word for Python

Special Tokens

Token	Behavior

`. : ;`	Self-delimiting (no whitespace needed)
`#`	Comment to end of line
`"..."`	String literal (auto-converts to String type)

Appendix: The Struggle with Return Stacks

ActorForth has one data stack. The traditional Forth model has two — a data stack and a return stack. Reconciling that difference was real work during the language's design, because a return stack isn't obviously wrong; it is a coherent answer to a coherent set of problems. This appendix records what a return stack does, what ActorForth does instead, and the specific reasons the language came out on the other side of the choice.

What a return stack does in a traditional Forth

Six jobs, ordered roughly from most to least mechanical:

1. **Return addresses for word calls.** When `word1` calls `word2`, the return stack holds the address where control resumes after `word2` completes.
2. **Temporary value stash.** `>r … r>` moves a value aside during an inner computation so the data stack can be reshaped without losing it.
3. **Loop counters.** `DO … LOOP` keeps limit and index on the return stack; `I` reads the current index.
4. **Locals.** Some Forths use the return stack as a frame for named-local values.
5. **Exception unwinding.** `CATCH` / `THROW` tracks return-stack depth to know how far to unwind.
6. **Implementation stash** — any word holding something out of the way while it reaches elsewhere.

What ActorForth does instead

ActorForth currently runs on the BEAM. The BEAM has its own call stack with tail-call optimization, proper overflow semantics, and process-isolated frames. Every a4 word call is an Erlang function call; control transfer and return are handled by that mechanism.

For the other five jobs:

1. **Return addresses** — BEAM's call stack, with proper TCO.
2. **Temporary stash** — currently stack juggling (`dup`, `swap`, `rot`, `over`). Open to improvement via combinators and auto-field bindings for product types.
3. **Loop counters** — tail-recursive word definitions with value-constrained sub-clauses (`: countdown Int -> ; : 0 -> ; drop : Int -> ; 1 - countdown .`). Typed, arbitrary termination conditions, clean return.
4. **Locals** — auto-field bindings on product-type inputs; named parameters as a planned extension.
5. **Exception unwinding** — `#af_error{}` + Erlang `try` / `catch`, with a `word_trace` field for call-chain context.
6. **Implementation stash** — closures capture what they need.

Why a4 does not expose a return stack

Three specific reasons, load-bearing:

1. **Adding a user-visible return stack adds a second stack picture to every word's signature.** ActorForth already asks a word to declare its data-stack effect as `InputTypes -> OutputTypes`. Exposing a return stack requires words to also declare their effect on it — or accept that such effects are unchecked. Either option roughly doubles the cognitive load of reading or writing a word signature. The single-stack model keeps the signature story a single question: what types move across the data stack?

2. **Values on a return stack would lack the type discipline every other value in a4 has.** Everything on the data stack is a `{Type, Value}` pair; dispatch, type checking, pattern matching, and error reporting all lean on that. A return stack carrying arbitrary transient values (return addresses, loop counters, user stashes) either needs a parallel type system or becomes an escape hatch where untyped values can live. Either path introduces a category of runtime behavior that the rest of the language is deliberately designed to prevent.

3. **ActorForth intends to eventually target native hardware, including minimal FPGA substrates.** Keeping return-stack mechanics as an implementation detail — not part of the language's contract with user code — preserves the freedom for a future backend to choose whatever call/return mechanism fits the hardware. If the language committed to a user-visible return stack, every backend would have to provide one with the same semantics; not committing leaves the design space open.

The homoiconicity question

A natural worry: does depending on the BEAM's call stack weaken ActorForth's claim to homoiconicity?

Homoiconicity is a claim about the relationship between code and data — that code is represented as a data structure the language itself can manipulate. For a4 that relationship holds in the specific ways that matter:

- Source tokens and compiled word bodies share the representation `[#token{}]`.
- Types are data (`#af_type{}` records).
- The dictionary is data (a map).
- The compiler is four types with handlers; its state is on the data stack during compilation.

Where control-flow happens to be tracked during execution is a property of the runtime, not of the language. The code-as-data relationship is unchanged by it. Lisp is homoiconic and runs on C's call stack. The same argument applies here.

What a future native / FPGA target still has to do

Some mechanism has to track return addresses. A hardware return stack is a typical answer; a program-counter-plus-stack-frame mechanism in software is another; registers-plus-explicit-save/restore is another. Which one an a4 backend uses is that backend's business.

The point of not exposing the return stack in the language is that none of these implementation choices are forced on user-facing code. Users write the same word definitions regardless of whether the backend is BEAM today, LLVM tomorrow, or a four-primitive FPGA next year. That portability is what the language contract buys when it stays silent on the mechanism.

What was removed

The continuation record originally declared a `return_stack` field — unused. A full audit found that no code path in the language's first year of development ever wrote to it. It was read only in two places: a depth-tracking counter in the test DSL (which always showed zero) and a `capture-return` primitive (which always returned an empty list). Both were removed alongside the field itself. The test-DSL word `max-return-depth` went with them, since it would have been asserting against a value that could never change.

If a future version of a4 decides a user-visible return stack is the right answer for some specific problem, adding the field back is a small change. Keeping it declared but unused was not serving any purpose.

The temporary-stash gap

The one job the traditional return stack does more concisely than ActorForth currently does is **give you a place to set a value aside while you compute with others.** `>r … r>` is shorter than any dance of `dup`, `swap`, `rot`, `over` in a4 today.

This is real, and the answer is not to bring back the return stack for the reasons above. The answer is a small set of concatenative combinators that turn "set this aside" into a declarative annotation (`bi`, `tri`, `keep`) rather than an explicit stack operation, plus auto-field bindings on product-type inputs (already shipped, see Chapter 5). Both land as features on the data stack alone, with the language's existing type discipline.